

Search Functionality

Now we need to create the `search` method in the `JobController`. Open the `app/Http/Controllers/JobController.php` file and add the following method.

```
// @desc Search for jobs
// @route GET /jobs/search
public function search(Request $request)
{
    $keywords = strtolower($request->input('keywords'));
    $location = strtolower($request->input('location'));

    $query = Job::query();

    if ($keywords) {
        $query->where(function ($q) use ($keywords) {
            $q->whereRaw('LOWER(title) like ?', ['%' . $keywords . '%'])
                ->orWhereRaw('LOWER(description) like ?', ['%' . $keywords .
                '%']);
        });
    }

    if ($location) {
        $query->where(function ($q) use ($location) {
            $q->whereRaw('LOWER(address) like ?', ['%' . $location . '%'])
                ->orWhereRaw('LOWER(city) like ?', ['%' . $location . '%'])
                ->orWhereRaw('LOWER(state) like ?', ['%' . $location . '%'])
                ->orWhereRaw('LOWER(zipcode) like ?', ['%' . $location . '%']);
        });
    }

    $jobs = $query->paginate(12);

    return view('jobs.index')->with('jobs', $jobs);
}
```

Some of this may be confusing, such as `$query->where(function ($q) use ($keywords)`.

This is where the main query logic happens. The `where()` method is being called on the query builder object (`$query`).

Inside the `where()` method, a closure (anonymous function) is used. This closure allows you to encapsulate multiple conditions for the where clause.

`use ($keywords)` is used to pass variables from the parent scope (in this case, `$keywords`) into the closure. Without `use`, the closure wouldn't have access to the `$keywords` variable since it's defined outside of the closure.

We are using the `whereRaw` method to pass in raw SQL so that we can use `LOWER` to make the search case-insensitive.

We are using the `paginate` method to paginate the search results. We are passing the search results to the regular `jobs.index` view.

That's it! Now the search form should work. Try searching for a job and see if the search results are displayed on the page.

Keep Text Values in Search Form

Let's add a hidden input to the search form to keep the text values in the search form when the page is refreshed. Open the `/resources/views/components/search.blade.php` file and add the following values to the inputs:

```
<input
  type="text"
  name="keywords"
  placeholder="Keywords"
  class="w-full md:w-72 px-4 py-3 focus:outline-none"
  value="{{ request('keywords') }}"
/>
<input
  type="text"
  name="location"
  placeholder="Location"
  class="w-full md:w-72 px-4 py-3 focus:outline-none"
  value="{{ request('location') }}"
/>
```

Back Button

Let's add a back button if showing search results. In the

`resources/views/jobs/index.blade.php` file, add the following code just below the div that wraps the search component:

```
<!-- Back Button -->
@if(request()->has('keywords') || request()->has('location'))
<a
  href="{{ route('jobs.index') }}"
  class="bg-gray-700 hover:bg-gray-600 text-white px-4 py-2 rounded mb-4 inline-block"
>
  <i class="fa fa-arrow-left mr-1"></i> Back
</a>
@endif
```

This will check for any search parameters in the URL and if there are any, it will display a back button.